

Project Beginning

Spotify Top-50-World — Before You Run the Notebook

This document is your orientation before touching any code. Read it fully, complete the setup steps, and only then open the notebook. Everything here is meant to install the right mindset and the right groundwork — the actual analysis is something you'll discover for yourself, inside the notebook, not something pre-explained here.

1. Welcome to the Project

You're about to take one raw CSV file and turn it into a working, 5-page Power BI report, backed by a real cloud database. The path looks like this:

1. **Python / Jupyter notebook** — load, explore, clean, and reshape the raw data
2. **Supabase (a hosted Postgres database)** — where the data actually lives, not just on your laptop
3. **A normalized data model** — three related tables instead of one flat file
4. **DAX measures in Power BI** — the calculations your report is built on
5. **A 5-page Power BI report** — the final, presentable deliverable

Why start in Python, not Power BI?

Python gives you full, granular control to inspect data row by row and explain every single change you make. Power BI is where you present conclusions — not where you should be discovering problems for the first time.

2. The Raw Dataset

The file you're starting from is `spotify-top-50-world.csv` — a daily snapshot of Spotify's Top 50 (World) chart. Before opening it in code, know what each column is supposed to represent:

Column	Type	What it holds
date	date	The calendar day this chart snapshot was taken.
position	integer	The song's rank that day — expected range 1 to 50.

Column	Type	What it holds
song	text	The track title, as published.
artist	text	The performing artist or artist combination.
popularity	integer	Spotify's own popularity score for the track, documented as a 0-100 scale.
duration_ms	integer	Track length, in milliseconds.
album_type	text	Whether the track came from a "single" or a full "album".
total_tracks	integer	How many tracks are on the parent album or single.
release_date	text	The track's original release date, as published.
is_explicit	boolean	Whether the track is tagged as explicit content.
album_cover_url	text	A link to the track's cover art image.

Understanding Granularity

"Granularity" (or *grain*) means: what does a single row in this table actually represent? Get this wrong, and every count, average, and chart you build afterward will be wrong too — even if the code runs without errors.

This dataset's grain is: **one row per song, per chart day**. Not one row per song. Not one row per artist. A single song that stayed on the chart for 100 days contributes 100 separate rows to this file — one for each day it charted, at whatever position it held that day.

Think about this before you open the notebook

If the grain is "one row per song per day," what happens to a column like *artist* or *duration_ms* — values that don't actually change from day to day for the same song? They get repeated, identically, on every single row that song appears in. Keep that in mind for Section 3.

3. What Could Be Wrong With It?

Never assume a dataset is clean just because it opened without an error. A CSV pulled from an external source can carry problems that are completely invisible until you go looking for them. Before you write a single line of cleaning code, it helps to know the *categories* of things worth checking for — not because this specific dataset necessarily has all of them, but because this is the mindset that makes you a careful analyst rather than one who got lucky.

Duplication. Could the exact same logical record appear more than once? For a chart dataset, that might mean: could two rows claim the same song held the same position on the same day?

Denormalization. You just identified this dataset's grain as "one row per song per day." That means song-level details (artist, duration, album info) repeat across every single day that song charts. Repetition like this isn't just wasteful — it creates a real risk that the same fact gets recorded slightly differently on different rows.

Encoding & text corruption. Text fields pulled from external systems can carry corrupted characters, especially anywhere involving special symbols, accented letters, or non-English text. These often look completely normal at a glance and only reveal themselves under close inspection.

Unexpected values. Do numeric columns actually stay inside their documented range? A popularity score is supposed to be 0-100 — is it, everywhere? Are there values that are technically valid but suspicious, like a score of exactly zero?

Identifier safety. Does a column that looks like it uniquely identifies something actually do so? A song title alone might seem like a safe way to identify a track — until two completely different artists happen to have released songs with the same name.

Format inconsistency. Are structured fields like dates stored the same way in every row? A date column can quietly contain a mix of full dates, month-only values, or year-only values, all sitting in the same column.

The Checks Mindset

For every category above, there's a concrete way to check rather than assume: count nulls, count duplicates, compare distinct counts against what you'd logically expect, check min/max against documented bounds, and actually look at real value samples instead of trusting a summary statistic alone. A check that quietly passes doesn't always mean the data is clean — sometimes it means the check itself wasn't specific enough to catch the real problem.

What to expect once you start

You will very likely find real issues once you start checking. That's the process working correctly, not a failure on your part or a broken dataset. The goal isn't a dataset with zero problems — it's a dataset where every problem has been found, understood, and either fixed with a documented reason, or knowingly left as an open question.

4. The Notebook's Purpose

The notebook you're about to open follows an **ELT** approach — Extract, Load, *then* Transform — rather than cleaning data before it ever reaches the database. In practice, that means:

1. Load the raw CSV into memory — exactly as published, untouched.
2. Connect securely to your Supabase database (setup steps below).
3. Upload the raw data as-is, before any cleaning. This preserves an untouched original you can always return to.
4. Explore the data — this is where you'll apply the checks mindset from Section 3 for real, against the real dataset.
5. Apply fixes — each one tied to a specific thing you actually found, with your reasoning written alongside it. Nothing gets changed silently.
6. Reshape the data into a proper, normalized model and upload the final version back to Supabase.

Why this order matters

If a cleaning decision later turns out to be wrong, you always have the untouched original sitting safely in the database to go back to. Nothing is ever silently overwritten without a version you could recover.

5. What You'll Need Before You Start

- A **Supabase account and a new project** — this becomes your live cloud database.
- A **credentials file**, in a specific JSON format, saved in the correct location — covered in Sections 6 and 7 below.
- Python with the required libraries installed: `pandas`, `sqlalchemy`, `psycopg2-binary`. Install once, before your first run: `pip install pandas sqlalchemy psycopg2-binary`

6. Create Your Supabase Project

1. Go to `supabase.com` and sign in, or create a free account if you don't already have one.
2. Click **New Project**.
3. Choose your organization, and give the project a name (e.g. `spotify-top-50-world`).
4. Set a **Database Password**. You can let Supabase generate one, but copy it down immediately — it may not be shown to you again. This becomes one of the five values in your credentials file.
5. Choose a region, then click **Create new project**.
6. Wait for provisioning to finish — this typically takes a minute or two.
7. Once the project is ready, click the **Connect** button near the top of the project dashboard.
8. In the connection options, select the **Session pooler** string (not Direct connection, not Transaction pooler) — this is the mode this project's notebook is built to use.
9. From the connection details shown, note down: **Host**, **Port** (5432 for session mode), **Database name** (normally `postgres`), and **User** (looks like `postgres.xxxxxxxxxxxx`).

A note on the database password

If you ever lose track of your database password, you can reset it from the project's Database Settings page — but resetting it will break any credentials file that still has the old password saved, so you'd need to update that file too.

7. Build Your `credentials.json`

The notebook reads your database connection details from a separate JSON file, rather than asking you to type them directly into a code cell. This keeps your password out of the notebook's own saved history. It expects exactly these five keys:

```
{  
  "db_host": "your-project-pooler-host",  
  "db_port": 5432,  
  "db_name": "postgres",  
  "db_user": "your-pooler-username",  
  "db_password": "your-database-password"  
}
```

Fill in the five values using what you noted down in Section 6, then save this file **in the same folder as your notebook and CSV file** — your project folder. The notebook will prompt you for this file's exact path the first time you run it.

Keep this file private

Never commit this file to version control (add it to `.gitignore` if the folder is ever put under Git), never paste its contents into a chat, and never share it outside your own machine. Anyone with this file has direct access to your database.

8. Final Checklist — Green Light

Before you open the notebook and run the first cell, confirm every item below:

- Project folder created, with the notebook (.ipynb), the CSV, and credentials.json all sitting together in it.
- Supabase project created and fully provisioned (not still "setting up").
- credentials.json filled in with all five correct values, saved in the project folder.
- Required Python libraries installed: pandas, sqlalchemy, psycopg2-binary.
- You understand the ELT approach, what "grain" means for this dataset, and the categories of flaws worth checking for before trusting any data.

Once every box is checked — you're ready. Open the notebook, and run the first cell. From here, let the notebook itself guide you: every finding it surfaces is real, and every fix you make should be something you could explain out loud, in one sentence, to someone else.